

第 10 章 ROS进阶

在本教程的第二章内容介绍了ROS的核心实现:通信机制——话题通信、服务通信和参数服务器。三者结合可以满足ROS中的大多数数据传输相关的应用场景，但是在一些特定场景下可能就有些力不从心了，本章主要介绍之前的通信机制存在的问题以及对应的优化策略，本章主要内容如下：

- action通信；
- 动态参数；
- pluginlib；
- nodelet。

本章预期达成的学习目标：

- 了解服务通信应用的局限性(action的应用场景)，熟练掌握action的理论模型与实现流程；
- 了解参数服务器应用的局限性(动态配置参数的应用场景)，熟练掌握动态配置参数的实现流程；
- 了解插件的概念以及使用流程；
- 了解nodelet的应用场景以及使用流程。

10.1 action通信

关于action通信，我们先从之前导航中的应用场景开始介绍，描述如下：

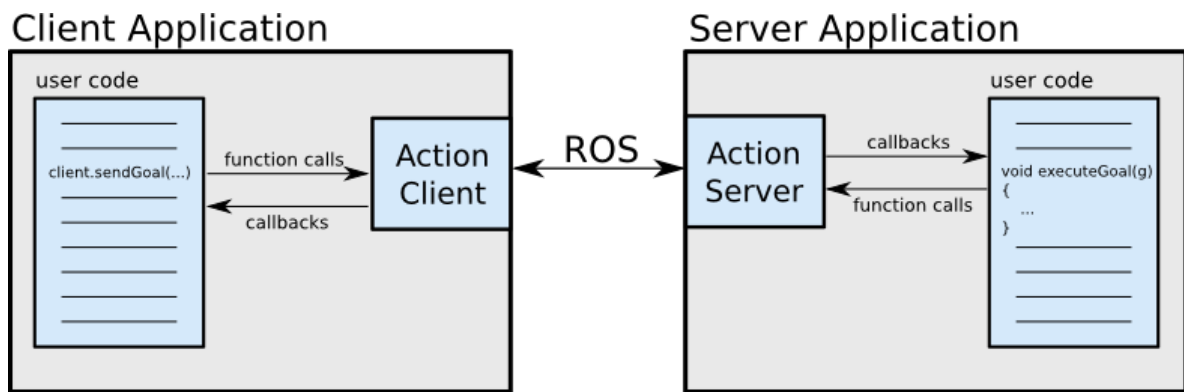
机器人导航到某个目标点,此过程需要一个节点A发布目标信息，然后一个节点B接收到请求并控制移动，最终响应目标达成状态信息。

乍一看，这好像是服务通信实现，因为需求中要A发送目标，B执行并返回结果，这是一个典型的基于请求响应的应答模式，不过，如果只是使用基本的服务通信实现，存在一个问题：导航是一个过程，是耗时操作，如果使用服务通信，那么只有在导航结束时，才会产生响应结果，而在导航过程中，节点A是不会获取到任何反馈的，从而可能出现程序“假死”的现象，过程的不可控意味着不良的用户体验，以及逻辑处理的缺陷(比如:导航中止的需求无法实现)。更合理的方案应该是:导航过程中，可以连续反馈当前机器人状态信息，当导航终止时，再返回最终的执行结果。在ROS中，该实现策略称之为:action 通信。

概念

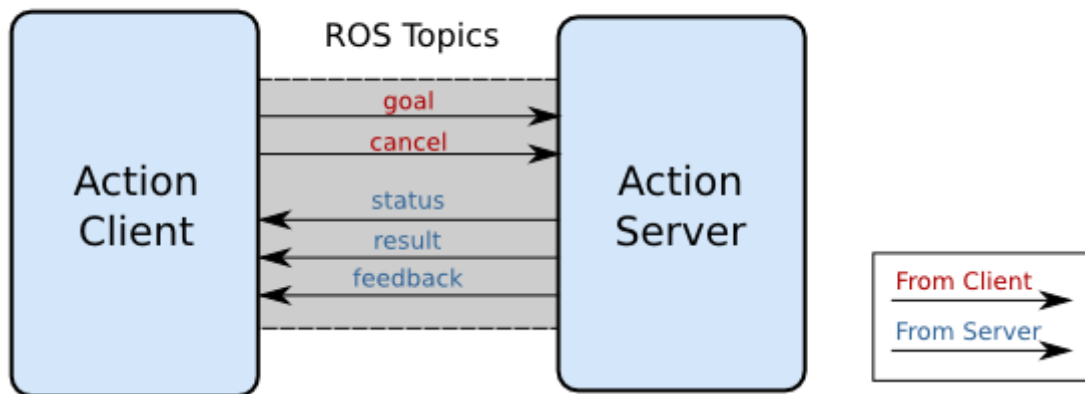
在ROS中提供了actionlib功能包集，用于实现 action 通信。action 是一种类似于服务通信的实现，其实现模型也包含请求和响应，但是不同的是，在请求和响应的过程中，服务端还可以连续的反馈当前任务进度，客户端可以接收连续反馈并且还可以取消任务。

action结构图解：



action通信接口图解:

Action Interface



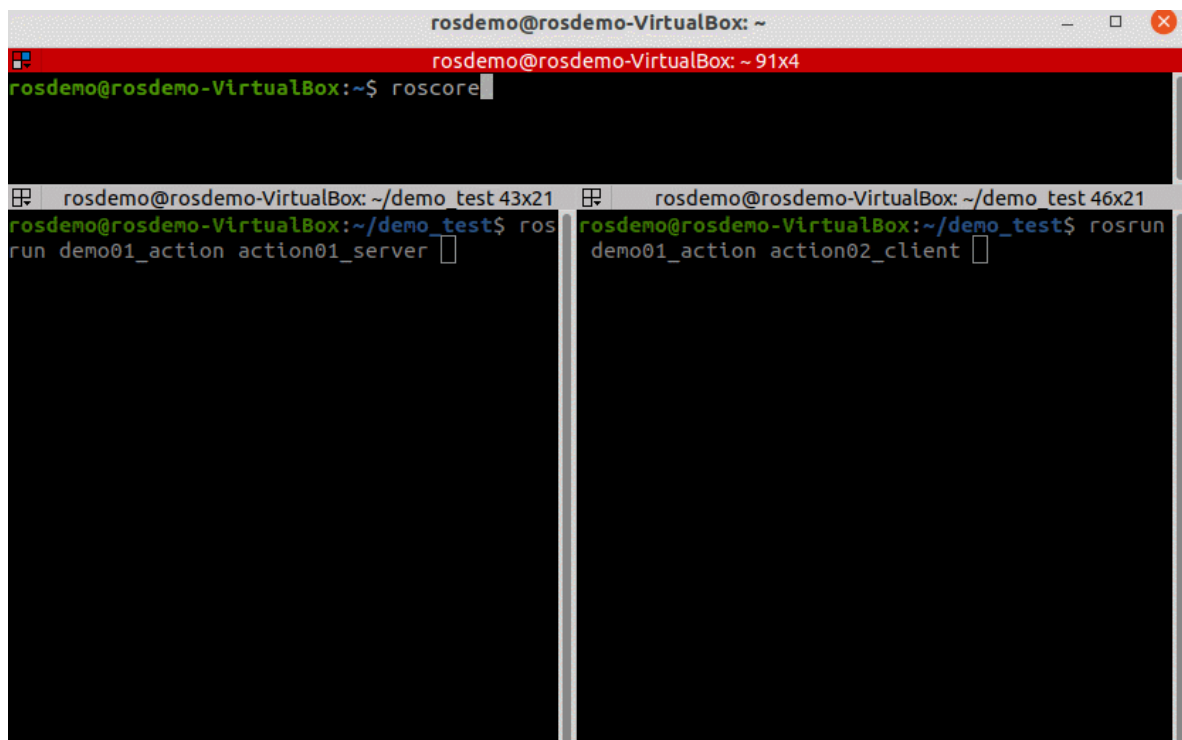
- goal:目标任务;
- cancel:取消任务;
- status:服务端状态;
- result:最终执行结果(只会发布一次);
- feedback:连续反馈(可以发布多次)。

作用

一般适用于耗时的请求响应场景,用以获取连续的状态反馈。

案例

创建两个ROS 节点, 服务器和客户端, 客户端可以向服务器发送目标数据N(一个整型数据)服务器会计算 1 到 N 之间所有整数的和,这是一个循环累加的过程, 返回给客户端, 这是基于请求响应模式的, 又已知服务器从接收到请求到产生响应是一个耗时操作, 每累加一次耗时0.1s, 为了良好的用户体验, 需要服务器在计算过程中, 每累加一次, 就给客户端响应一次百分比格式的执行进度, 使用 action实现。



另请参考:

- <http://wiki.ros.org/actionlib>
- http://wiki.ros.org/actionlib_tutorials/Tutorials

10.1.1 action通信自定义action文件

action、srv、msg 文件内的可用数据类型一致，且三者实现流程类似:

1. 按照固定格式创建action文件;
2. 编辑配置文件;
3. 编译生成中间文件。

1. 定义action文件

首先新建功能包，并导入依赖: `roscpp rospy std_msgs actionlib actionlib_msgs` ;

然后功能包下新建 action 目录，新增 Xxx.action(比如:AddInts.action)。

action 文件内容组成为三部分:请求目标值、最终响应结果、连续反馈，三者之间使用 `---` 分割
示例内容如下:

```
1 | #目标值
2 | int32 num
3 | ---
4 | #最终结果
5 | int32 result
6 | ---
7 | #连续反馈
8 | float64 progress_bar
```

2.编辑配置文件

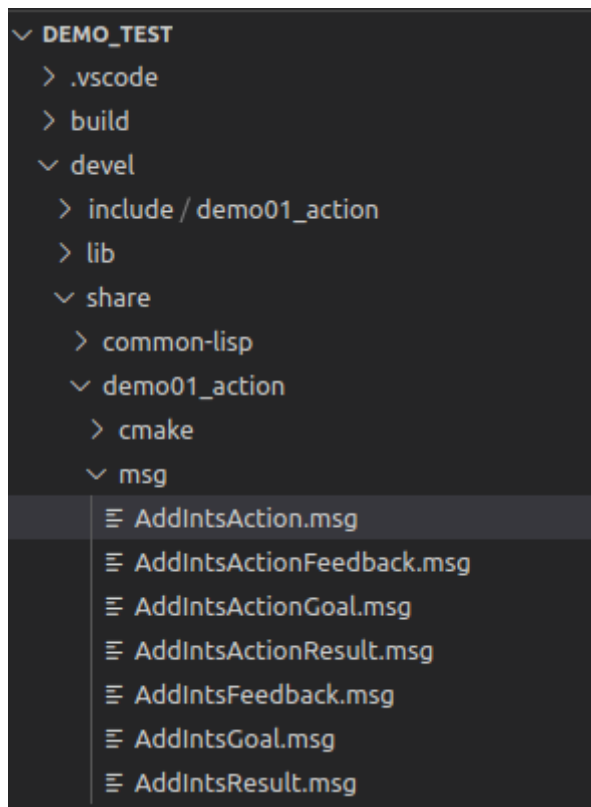
CMakeLists.txt

```
1 find_package
2 (catkin REQUIRED COMPONENTS
3   roscpp
4   rospy
5   std_msgs
6   actionlib
7   actionlib_msgs
8 )
9 Copy
10 add_action_files(
11   FILES
12   AddInts.action
13 )
14 Copy
15 generate_messages(
16   DEPENDENCIES
17   std_msgs
18   actionlib_msgs
19 )
20 Copy
21 catkin_package(
22
23   # INCLUDE_DIRS include
24   # LIBRARIES demo04_action
25
26   CATKIN_DEPENDS roscpp rospy std_msgs actionlib actionlib_msgs
27
28   # DEPENDS system_lib
29
30 )
```

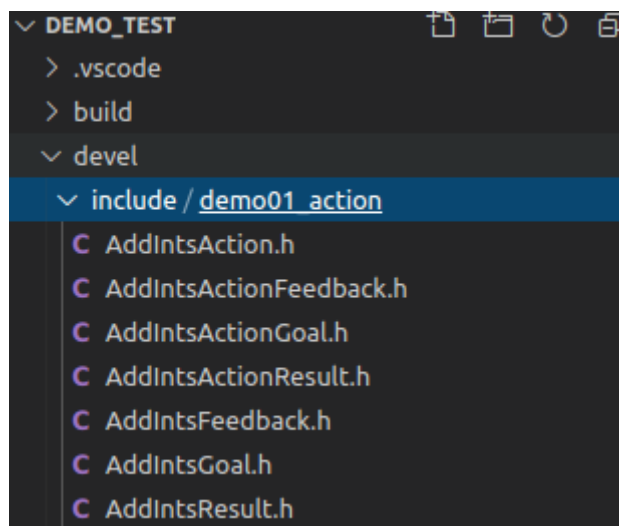
3.编译

编译后会生成一些中间文件。

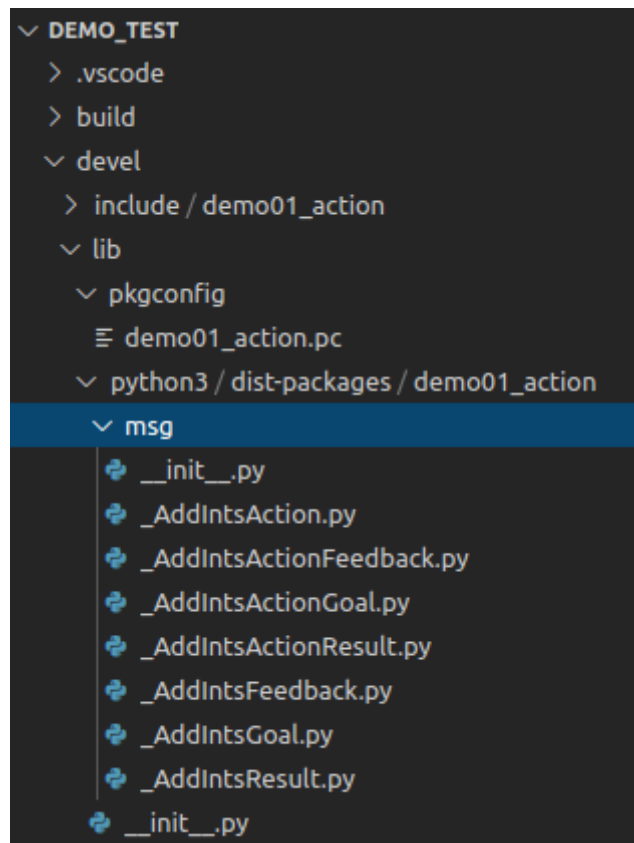
msg文件(.../工作空间/devel/share/包名/msg/xxx.msg):



C++ 调用的文件(.../工作空间/devel/include/包名/xxx.h):



Python 调用的文件(.../工作空间/devel/lib/python3/dist-packages/包名/msg/xxx.py):



10.1.2 action通信自定义action文件调用A(C++)

需求:

创建两个ROS 节点，服务器和客户端，客户端可以向服务器发送目标数据N(一个整型数据)服务器会计算 1 到 N 之间所有整数的和,这是一个循环累加的过程，返回给客户端，这是基于请求响应模式的，又已知服务器从接收到请求到产生响应是一个耗时操作，每累加一次耗时0.1s，为了良好的用户体验，需要服务器在计算过程中，每累加一次，就给客户端响应一次百分比格式的执行进度，使用 action实现。

流程:

1. 编写action服务端实现;
2. 编写action客户端实现;
3. 编辑CMakeLists.txt;
4. 编译并执行。

0.vscode配置

需要像之前自定义 msg 实现一样配置c_cpp_properties.json 文件，如果以前已经配置且没有变更工作空间，可以忽略，如果需要配置，配置方式与之前相同:

```
1 {
2     "configurations": [
3         {
4             "browse": {
5                 "databaseFilename": "",
```

```

6         "limitSymbolsToIncludedHeaders": true
7     },
8     "includePath": [
9         "/opt/ros/noetic/include/**",
10        "/usr/include/**",
11        "/xxx/yyy工作空间/devel/include/**" //配置 head 文件的路径
12    ],
13    "name": "ROS",
14    "intelliSenseMode": "gcc-x64",
15    "compilerPath": "/usr/bin/gcc",
16    "cStandard": "c11",
17    "cppStandard": "c++17"
18    }
19 ],
20 "version": 4
21 }

```

1.服务端

```

1 #include "ros/ros.h"
2 #include "actionlib/server/simple_action_server.h"
3 #include "demo01_action/AddIntsAction.h"
4 /*
5     需求:
6         创建两个ROS节点, 服务器和客户端,
7         客户端可以向服务器发送目标数据N (一个整型数据)
8         服务器会计算1到N之间所有整数的和, 这是一个循环累加的过程, 返回给客户端,
9         这是基于请求响应模式的,
10        又已知服务器从接收到请求到产生响应是一个耗时操作, 每累加一次耗时0.1s,
11        为了良好的用户体验, 需要服务器在计算过程中,
12        每累加一次, 就给客户端响应一次百分比格式的执行进度, 使用action实现。
13
14    流程:
15        1.包含头文件;
16        2.初始化ROS节点;
17        3.创建NodeHandle;
18        4.创建action服务对象;
19        5.处理请求,产生反馈与响应;
20        6.spin().
21
22 */
23
24 typedef actionlib::SimpleActionServer<demo01_action::AddIntsAction> Server;
25
26
27 void cb(const demo01_action::AddIntsGoalConstPtr &goal, Server* server){
28     //获取目标值
29     int num = goal->num;
30     ROS_INFO("目标值:%d", num);
31     //累加并响应连续反馈
32     int result = 0;

```

```

33     demo01_action::AddIntsFeedback feedback; //连续反馈
34     ros::Rate rate(10); //通过频率设置休眠时间
35     for (int i = 1; i <= num; i++)
36     {
37         result += i;
38         //组织连续数据并发布
39         feedback.progress_bar = i / (double)num;
40         server->publishFeedback(feedback);
41         rate.sleep();
42     }
43     //设置最终结果
44     demo01_action::AddIntsResult r;
45     r.result = result;
46     server->setSucceeded(r);
47     ROS_INFO("最终结果:%d", r.result);
48 }
49
50 int main(int argc, char *argv[])
51 {
52     setlocale(LC_ALL, "");
53     ROS_INFO("action服务端实现");
54     // 2.初始化ROS节点;
55     ros::init(argc, argv, "AddInts_server");
56     // 3.创建NodeHandle;
57     ros::NodeHandle nh;
58     // 4.创建action服务对象;
59     /*SimpleActionServer(ros::NodeHandle n,
60                          std::string name,
61                          boost::function<void (const
demo01_action::AddIntsGoalConstPtr &)> execute_callback,
62                          bool auto_start)
63     */
64     // actionlib::SimpleActionServer<demo01_action::AddIntsAction>
server(...);
65     Server server(nh, "addInts", boost::bind(&cb, _1, &server), false);
66     server.start();
67     // 5.处理请求,产生反馈与响应;
68
69     // 6.spin().
70     ros::spin();
71     return 0;
72 }

```

PS:

可以先配置CMakeLists.tx文件并启动上述action服务端，然后通过 `rostopic` 查看话题，向action相关话题发送消息，或订阅action相关话题的消息。

2.客户端

```

1  #include "ros/ros.h"
2  #include "actionlib/client/simple_action_client.h"

```



```

3  #include "demo01_action/AddIntsAction.h"
4
5  /*
6      需求:
7          创建两个ROS节点, 服务器和客户端,
8          客户端可以向服务器发送目标数据N (一个整型数据)
9          服务器会计算1到N之间所有整数的和, 这是一个循环累加的过程, 返回给客户端,
10         这是基于请求响应模式的,
11         又已知服务器从接收到请求到产生响应是一个耗时操作, 每累加一次耗时0.1s,
12         为了良好的用户体验, 需要服务器在计算过程中,
13         每累加一次, 就给客户端响应一次百分比格式的执行进度, 使用action实现。
14
15     流程:
16         1.包含头文件;
17         2.初始化ROS节点;
18         3.创建NodeHandle;
19         4.创建action客户端对象;
20         5.发送目标, 处理反馈以及最终结果;
21         6.spin().
22
23     */
24     typedef actionlib::SimpleActionClient<demo01_action::AddIntsAction> Client;
25
26
27     //处理最终结果
28     void done_cb(const actionlib::SimpleClientGoalState &state, const
demo01_action::AddIntsResultConstPtr &result){
29         if (state.state_ == state.SUCCEEDED)
30         {
31             ROS_INFO("最终结果:%d",result->result);
32         } else {
33             ROS_INFO("任务失败! ");
34         }
35     }
36 }
37 //服务已经激活
38 void active_cb(){
39     ROS_INFO("服务已经被激活....");
40 }
41 //处理连续反馈
42 void feedback_cb(const demo01_action::AddIntsFeedbackConstPtr &feedback){
43     ROS_INFO("当前进度:%.2f",feedback->progress_bar);
44 }
45
46
47 int main(int argc, char *argv[])
48 {
49     setlocale(LC_ALL,"");
50     // 2.初始化ROS节点;
51     ros::init(argc,argv,"AddInts_client");
52     // 3.创建NodeHandle;
53     ros::NodeHandle nh;

```

```

54 // 4.创建action客户端对象;
55 // SimpleActionClient(ros::NodeHandle & n, const std::string & name, bool
spin_thread = true)
56 // actionlib::SimpleActionClient<demo01_action::AddIntsAction>
client(nh,"addInts");
57 Client client(nh,"addInts",true);
58 //等待服务启动
59 client.waitForServer();
60 // 5.发送目标, 处理反馈以及最终结果;
61 /*
62 void sendGoal(const demo01_action::AddIntsGoal &goal,
63 boost::function<void (const actionlib::SimpleClientGoalState
&state, const demo01_action::AddIntsResultConstPtr &result)> done_cb,
64 boost::function<void ()> active_cb,
65 boost::function<void (const
demo01_action::AddIntsFeedbackConstPtr &feedback)> feedback_cb)
66
67 */
68 demo01_action::AddIntsGoal goal;
69 goal.num = 10;
70
71 client.sendGoal(goal,&done_cb,&active_cb,&feedback_cb);
72 // 6.spin().
73 ros::spin();
74 return 0;
75 }

```

PS:等待服务启动, 只可以使用 `client.waitForServer();`, 之前服务中等待启动的另一种方式 `ros::service::waitForService("addInts");` 不适用

3.编译配置文件

```

1 add_executable(action01_server src/action01_server.cpp)
2 add_executable(action02_client src/action02_client.cpp)
3 ...
4
5 add_dependencies(action01_server ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})
6 add_dependencies(action02_client ${${PROJECT_NAME}_EXPORTED_TARGETS}
${catkin_EXPORTED_TARGETS})
7 ...
8
9 target_link_libraries(action01_server
10 ${catkin_LIBRARIES}
11 )
12 target_link_libraries(action02_client
13 ${catkin_LIBRARIES}
14 )

```

4.执行

首先启动 roscore，然后分别启动action服务端与action客户端，最终运行结果与案例类似。

10.1.3 action通信自定义action文件调用(Python)

需求:

创建两个ROS 节点，服务器和客户端，客户端可以向服务器发送目标数据N(一个整型数据)服务器会计算 1 到 N 之间所有整数的和,这是一个循环累加的过程，返回给客户端，这是基于请求响应模式的，又已知服务器从接收到请求到产生响应是一个耗时操作，每累加一次耗时0.1s，为了良好的用户体验，需要服务器在计算过程中，每累加一次，就给客户端响应一次百分比格式的执行进度，使用 action实现。

流程:

1. 编写action服务端实现;
2. 编写action客户端实现;
3. 编辑CMakeLists.txt;
4. 编译并执行。

0.vscode配置

需要像之前自定义 msg 实现一样配置settings.json 文件，如果以前已经配置且没有变更工作空间，可以忽略，如果需要配置，配置方式与之前相同:

```
1  {
2      "python.autoComplete.extraPaths": [
3          "/opt/ros/noetic/lib/python3/dist-packages",
4          "/xxx/yyy工作空间/devel/lib/python3/dist-packages"
5      ]
6  }
```

1.服务端

```
1  #! /usr/bin/env python
2  import rospy
3  import actionlib
4  from demo01_action.msg import *
5  """
6      需求:
7          创建两个ROS 节点，服务器和客户端，
8          客户端可以向服务器发送目标数据N(一个整型数据)服务器会计算 1 到 N 之间所有整
9          数的和，
10         这是一个循环累加的过程，返回给客户端，这是基于请求响应模式的，
11         又已知服务器从接收到请求到产生响应是一个耗时操作，每累加一次耗时0.1s，
12         为了良好的用户体验，需要服务器在计算过程中，
13         每累加一次，就给客户端响应一次百分比格式的执行进度，使用 action实现。
14     流程:
15         1. 导包
```

```

15         2.初始化 ROS 节点
16         3.使用类封装, 然后创建对象
17         4.创建服务器对象
18         5.处理请求数据产生响应结果, 中间还要连续反馈
19         6.spin
20
21
22 class MyActionServer:
23     def __init__(self):
24         #SimpleActionServer(name, ActionSpec, execute_cb=None,
25         auto_start=True)
26         self.server =
27         actionlib.SimpleActionServer("addInts",AddIntsAction,self.cb,False)
28         self.server.start()
29         rospy.loginfo("服务端启动")
30
31     def cb(self,goal):
32         rospy.loginfo("服务端处理请求:")
33         #1.解析目标值
34         num = goal.num
35         #2.循环累加, 连续反馈
36         rate = rospy.Rate(10)
37         sum = 0
38         for i in range(1,num + 1):
39             # 累加
40             sum = sum + i
41             # 计算进度并连续反馈
42             feedBack = i / num
43             rospy.loginfo("当前进度:%.2f",feedBack)
44
45             feedBack_obj = AddIntsFeedback()
46             feedBack_obj.progress_bar = feedBack
47             self.server.publish_feedback(feedBack_obj)
48             rate.sleep()
49         #3.响应最终结果
50         result = AddIntsResult()
51         result.result = sum
52         self.server.set_succeeded(result)
53         rospy.loginfo("响应结果:%d",sum)
54
55 if __name__ == "__main__":
56     rospy.init_node("action_server_p")
57     server = MyActionServer()
58     rospy.spin()

```

PS:

可以先配置CMakeLists.tx文件并启动上述action服务端, 然后通过 rostopic 查看话题, 向action 相关话题发送消息, 或订阅action相关话题的消息。

2.客户端

```

1  #!/usr/bin/env python
2
3  import rospy
4  import actionlib
5  from demo01_action.msg import *
6
7  """
8      需求:
9          创建两个ROS 节点, 服务器和客户端,
10         客户端可以向服务器发送目标数据N(一个整型数据)服务器会计算 1 到 N 之间所有整
11         数的和,
12         这是一个循环累加的过程, 返回给客户端, 这是基于请求响应模式的,
13         又已知服务器从接收到请求到产生响应是一个耗时操作, 每累加一次耗时0.1s,
14         为了良好的用户体验, 需要服务器在计算过程中,
15         每累加一次, 就给客户端响应一次百分比格式的执行进度, 使用 action实现。
16
17     流程:
18         1.导包
19         2.初始化 ROS 节点
20         3.创建 action Client 对象
21         4.等待服务
22         5.组织目标对象并发送
23         6.编写回调, 激活、连续反馈、最终响应
24         7.spin
25
26 """
27
28 def done_cb(state,result):
29     if state == actionlib.GoalStatus.SUCCEEDED:
30         rospy.loginfo("响应结果:%d",result.result)
31
32 def active_cb():
33     rospy.loginfo("服务被激活....")
34
35 def fb_cb(fb):
36     rospy.loginfo("当前进度:%.2f",fb.progress_bar)
37
38 if __name__ == "__main__":
39     # 2.初始化 ROS 节点
40     rospy.init_node("action_client_p")
41     # 3.创建 action Client 对象
42     client = actionlib.SimpleActionClient("addInts",AddIntsAction)
43     # 4.等待服务
44     client.wait_for_server()
45     # 5.组织目标对象并发送
46     goal_obj = AddIntsGoal()
47     goal_obj.num = 10
48     client.send_goal(goal_obj,done_cb,active_cb,fb_cb)
49     # 6.编写回调, 激活、连续反馈、最终响应
50     # 7.spin
51     rospy.spin()

```

3.编辑配置文件

先为 Python 文件添加可执行权限: `chmod +x *.py`

```
1 catkin_install_python(PROGRAMS
2   scripts/action01_server_p.py
3   scripts/action02_client_p.py
4   DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
5 )
```

4.执行

首先启动 `roscore`，然后分别启动action服务端与action客户端，最终运行结果与案例类似。

10.2动态参数

参数服务器的数据被修改时，如果节点不重新访问，那么就不能获取修改后的数据，例如在乌龟背景色修改的案例中，先启动乌龟显示节点，然后再修改参数服务器中关于背景色设置的参数，那么窗体的背景色是不会修改的，必须要重启乌龟显示节点才能生效。而一些特殊场景下，是要求能做到动态获取的，也即，参数一旦修改，能够通知节点参数已经修改并读取修改后的数据，比如：

机器人调试时，需要修改机器人轮廓信息(长宽高)、传感器位姿信息....，如果这些信息存储在参数服务器中，那么意味着需要重启节点，才能使更新设置生效，但是希望修改完毕之后，某些节点能够即时更新这些参数信息。

在ROS中针对这种场景已经给出的解决方案: `dynamic reconfigure` 动态配置参数。

动态配置参数，之所以能够实现即时更新，因为被设计成 CS 架构，客户端修改参数就是向服务器发送请求，服务器接收到请求之后，读取修改后的是参数。

概念

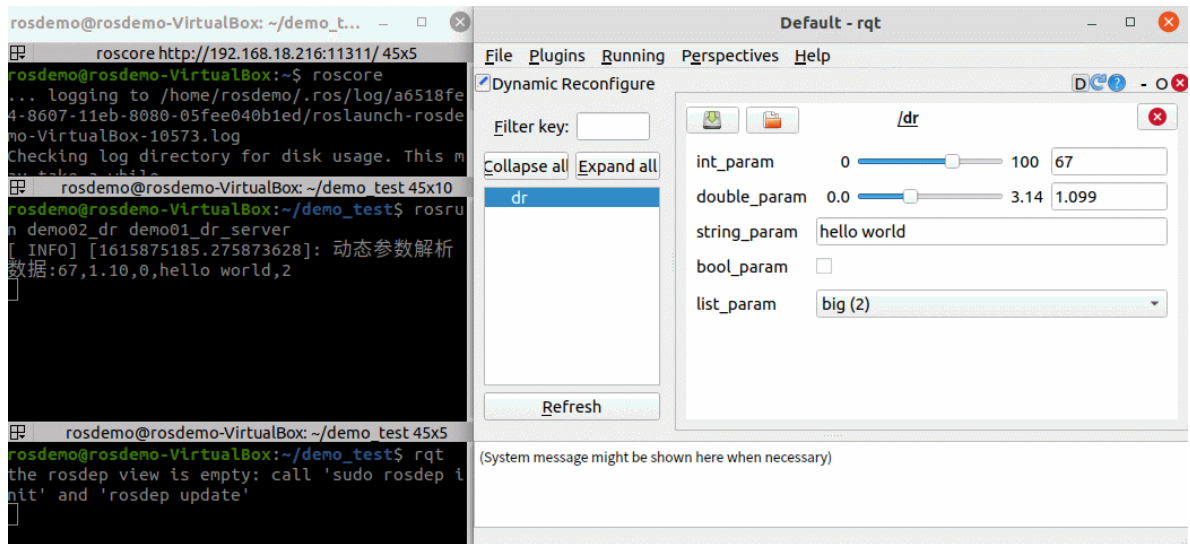
一种可以在运行时更新参数而无需重启节点的参数配置策略。

作用

主要应用于需要动态更新参数的场景，比如参数调试、功能切换等。典型应用:导航时参数的动态调试。

案例

编写两个节点，一个节点可以动态修改参数，另一个节点时时解析修改后的数据。



另请参考：

- http://wiki.ros.org/dynamic_reconfigure
- http://wiki.ros.org/dynamic_reconfigure/Tutorials

10.2.1 动态参数客户端

需求：

编写两个节点，一个节点可以动态修改参数，另一个节点时时解析修改后的数据。

客户端实现流程：

- 新建并编辑 .cfg 文件;
- 编辑CMakeLists.txt;
- 编译。

1.新建功能包

新建功能包，添加依赖: `roscpp rospy std_msgs dynamic_reconfigure` 。

2.添加.cfg文件

新建 cfg 文件夹，添加 xxx.cfg 文件(并添加可执行权限)，cfg 文件其实就是一个 python 文件,用于生成参数修改的客户端(GUI)。

```
1  #! /usr/bin/env python
2  """
3  4生成动态参数 int,double,bool,string,列表
4  5实现流程：
5  6    1.导包
6  7    2.创建生成器
7  8    3.向生成器添加若干参数
8  9    4.生成中间文件并退出
```

```

9  10
10  """
11  # 1.导包
12  from dynamic_reconfigure.parameter_generator_catkin import *
13  PACKAGE = "demo02_dr"
14  # 2.创建生成器
15  gen = ParameterGenerator()
16
17  # 3.向生成器添加若干参数
18  #add(name, paramtype, level, description, default=None, min=None, max=None,
19  edit_method="")
20  gen.add("int_param",int_t,0,"整型参数",50,0,100)
21  gen.add("double_param",double_t,0,"浮点参数",1.57,0,3.14)
22  gen.add("string_param",str_t,0,"字符串参数","hello world ")
23  gen.add("bool_param",bool_t,0,"bool参数",True)
24
25  many_enum = gen.enum([gen.const("small",int_t,0,"a small size"),
26                          gen.const("mediun",int_t,1,"a medium size"),
27                          gen.const("big",int_t,2,"a big size")
28                          ],"a car size set")
29
30  gen.add("list_param",int_t,0,"列表参数",0,0,2, edit_method=many_enum)
31
32  # 4.生成中间文件并退出
33  exit(gen.generate(PACKAGE,"dr_node","dr"))

```

`chmod +x xxx.cfg` 添加权限

3.配置 CMakeLists.txt

```

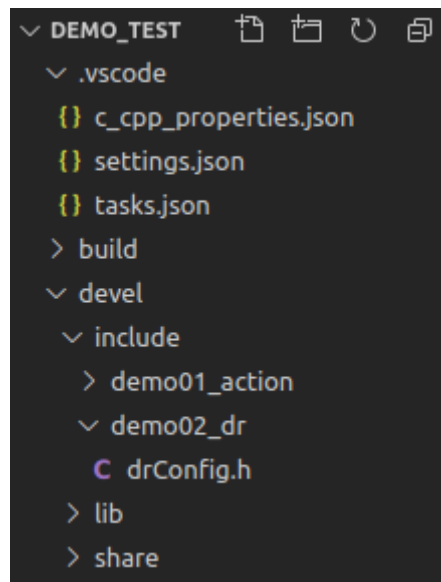
1  generate_dynamic_reconfigure_options(
2      cfg/mycar.cfg
3  )

```

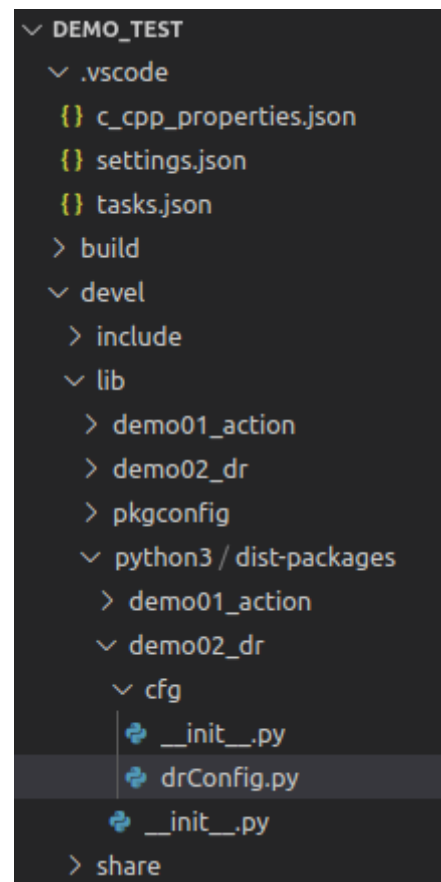
4.编译

编译后会生成中间文件

C++ 需要调用的头文件:



Python需要调用的文件:



10.2.2 动态参数服务端A(C++)

需求:

编写两个节点，一个节点可以动态修改参数，另一个节点时时解析修改后的数据。

服务端实现流程:

- 新建并编辑 c++ 文件;
- 编辑CMakeLists.txt;
- 编译并执行。

0.vscode配置

需要像之前自定义 msg 实现一样配置settings.json 文件，如果以前已经配置且没有变更工作空间，可以忽略，如果需要配置，配置方式与之前相同:

```
1 {
2     "configurations": [
3         {
4             "browse": {
5                 "databaseFilename": "",
6                 "limitSymbolsToIncludedHeaders": true
7             },
8             "includePath": [
9                 "/opt/ros/noetic/include/**",
10                "/usr/include/**",
11                "/xxx/yyy工作空间/devel/include/**" //配置 head 文件的路径
12            ],
13            "name": "ROS",
14            "intelliSenseMode": "gcc-x64",
15            "compilerPath": "/usr/bin/gcc",
16            "cStandard": "c11",
17            "cppStandard": "c++17"
18        }
19    ],
20    "version": 4
21 }
```

1.服务器代码实现

新建cpp文件，内容如下:

```
1 #include "ros/ros.h"
2 #include "dynamic_reconfigure/server.h"
3 #include "demo02_dr/drConfig.h"
4 /*
5     动态参数服务端：参数被修改时直接打印
6     实现流程：
7         1.包含头文件
8         2.初始化 ros 节点
9         3.创建服务器对象
10        4.创建回调对象(使用回调函数，打印修改后的参数)
11        5.服务器对象调用回调对象
12        6.spin()
13 */
14
15 void cb(demo02_dr::drConfig& config, uint32_t level){
16     ROS_INFO("动态参数解析数据:%d,%.2f,%d,%s,%d",
```

```

17         config.int_param,
18         config.double_param,
19         config.bool_param,
20         config.string_param.c_str(),
21         config.list_param
22     );
23 }
24
25 int main(int argc, char *argv[])
26 {
27     setlocale(LC_ALL, "");
28     // 2.初始化 ros 节点
29     ros::init(argc, argv, "dr");
30     // 3.创建服务器对象
31     dynamic_reconfigure::Server<demo02_dr::drConfig> server;
32     // 4.创建回调对象(使用回调函数, 打印修改后的参数)
33     dynamic_reconfigure::Server<demo02_dr::drConfig>::CallbackType cbType;
34     cbType = boost::bind(&cb, _1, _2);
35     // 5.服务器对象调用回调对象
36     server.setCallback(cbType);
37     // 6.spin()
38     ros::spin();
39     return 0;
40 }

```

2.编译配置文件

```

1 add_executable(demo01_dr_server src/demo01_dr_server.cpp)
2 ...
3
4 add_dependencies(demo01_dr_server ${${PROJECT_NAME}_EXPORTED_TARGETS}
5   ${catkin_EXPORTED_TARGETS})
6 ...
7
8 target_link_libraries(demo01_dr_server
9   ${catkin_LIBRARIES}
10 )

```

3.执行

先启动 `roscore`

启动服务端: `roslaunch 功能包 xxxx`

启动客户端: `roslaunch rqt_gui rqt_gui -s rqt_reconfigure` 或 `roslaunch rqt_reconfigure rqt_reconfigure`

最终可以通过客户端提供的界面修改数据, 并且修改完毕后, 服务端会即时输出修改后的结果, 最终运行结果与示例类似。

PS:ROS版本较新时，可能没有提供客户端相关的功能包导致 `roslaunch rqt_reconfigure rqt_reconfigure` 调用会抛出异常。

10.2.3 动态参数服务端B(Python)

需求:

编写两个节点，一个节点可以动态修改参数，另一个节点时时解析修改后的数据。

服务端实现流程:

- 新建并编辑 Python 文件;
- 编辑CMakeLists.txt;
- 编译并执行。

0.vscode配置

需要像之前自定义 msg 实现一样配置settings.json 文件，如果以前已经配置且没有变更工作空间，可以忽略，如果需要配置，配置方式与之前相同:

```
1 {
2     "python.autoComplete.extraPaths": [
3         "/opt/ros/noetic/lib/python3/dist-packages",
4         "/xxx/yyy工作空间/devel/lib/python3/dist-packages"
5     ]
6 }
```

1.服务器代码实现

新建python文件，内容如下:

```
1  #!/usr/bin/env python
2  import rospy
3  from dynamic_reconfigure.server import Server
4  from demo02_dr.cfg import drConfig
5
6  """
7      动态参数服务端：参数被修改时直接打印
8      实现流程：
9          1.导包
10         2.初始化 ros 节点
11         3.创建服务对象
12         4.回调函数处理
13         5.spin
14  """
15  # 回调函数
16  def cb(config, level):
17      rospy.loginfo("python 动态参数服务解析:%d,%.2f,%d,%s,%d",
```

```

18         config.int_param,
19         config.double_param,
20         config.bool_param,
21         config.string_param,
22         config.list_param
23     )
24     return config
25
26 if __name__ == "__main__":
27     # 2.初始化 ros 节点
28     rospy.init_node("dr_p")
29     # 3.创建服务对象
30     server = Server(drConfig,cb)
31     # 4.回调函数处理
32     # 5.spin
33     rospy.spin()

```

2.编辑配置文件

先为 Python 文件添加可执行权限: `chmod +x *.py`

```

1 catkin_install_python(PROGRAMS
2     scripts/demo01_dr_server_p.py
3     DESTINATION ${CATKIN_PACKAGE_BIN_DESTINATION}
4 )

```

3.执行

先启动 `roscore`

启动服务端: `roslaunch 功能包 xxxx.py`

启动客户端: `roslaunch rqt_gui rqt_gui -s rqt_reconfigure` 或 `roslaunch rqt_reconfigure rqt_reconfigure`

最终可以通过客户端提供的界面修改数据，并且修改完毕后，服务端会即时输出修改后的结果，最终运行结果与示例类似。

PS:ROS版本较新时，可能没有提供客户端相关的功能包导致 `roslaunch rqt_reconfigure rqt_reconfigure` 调用会抛出异常。

10.3 pluginlib

pluginlib直译是插件库，所谓插件字面意思就是可插拔的组件，比如:以计算机为例，可以通过USB接口自由插拔的键盘、鼠标、U盘...都可以看作是插件实现，其基本原理就是通过规范化的USB接口协议实现计算机与USB设备的自由组合。同理，在软件编程中，插件是一种遵循一定规范的应用程序接口编写出来的程序，插件程序依赖于某个应用程序，且应用程序可以与不同的插件程序自由组合。在ROS中，也会经常使用到插件，场景如下：

1.导航插件:在导航中,涉及到路径规划模块,路径规划算法有多种,也可以自实现,导航应用时,可能需要测试不同算法的优劣以选择更合适的实现,这种场景下,ROS中就是通过插件的方式来实现不同算法的灵活切换的。

2.rviz插件:在rviz中已经提供了丰富的功能实现,但是即便如此,特定场景下,开发者可能需要实现某些定制化功能并集成到rviz中,这一集成过程也是基于插件的。

概念

pluginlib是一个c++库,用来从一个ROS功能包中加载和卸载插件(plugin)。插件是指从运行时库中动态加载的类。通过使用Pluginlib,不必将某个应用程序显式地链接到包含某个类的库,Pluginlib可以随时打开包含类的库,而不需要应用程序事先知道包含类定义的库或者头文件。

作用

- 结构清晰;
- 低耦合,易修改,可维护性强;
- 可移植性强,更具复用性;
- 结构容易调整,插件可以自由增减;

另请参考:

- <http://wiki.ros.org/pluginlib>
- <http://wiki.ros.org/pluginlib/Tutorials/Writing%20and%20Using%20a%20Simple%20Plugin>

10.3.1 pluginlib使用

需求:

以插件的方式实现正多边形的相关计算。

实现流程:

1. 准备;
 2. 创建基类;
 3. 创建插件类;
 4. 注册插件;
 5. 构建插件库;
 6. 使插件可用于ROS工具链;
 - 配置xml
 - 导出插件
 7. 使用插件;
 8. 执行。
-

1.准备

创建功能包xxx导入依赖: roscpp pluginlib。

在 VSCode中需要配置 .vscode/c_cpp_properties.json文件中关于 includepath 选项的设置。

```
1  {
2      "configurations": [
3          {
4              "browse": {
5                  "databaseFilename": "",
6                  "limitSymbolsToIncludedHeaders": true
7              },
8              "includePath": [
9                  "/opt/ros/noetic/include/**",
10                 "/usr/include/**",
11                 "../yyy工作空间/功能包/include/**" //配置 head 文件的路径
12             ],
13             "name": "ROS",
14             "intelliSenseMode": "gcc-x64",
15             "compilerPath": "/usr/bin/gcc",
16             "cStandard": "c11",
17             "cppStandard": "c++17"
18         }
19     ],
20     "version": 4
21 }
```

2.创建基类

在 xxx/include/xxx下新建C++头文件: polygon_base.h, 所有的插件类都需要继承此基类, 内容如下:

```
1  #ifndef XXX_POLYGON_BASE_H_
2  #define XXX_POLYGON_BASE_H_
3
4  namespace polygon_base
5  {
6      class RegularPolygon
7      {
8      public:
9          virtual void initialize(double side_length) = 0;
10         virtual double area() = 0;
11         virtual ~RegularPolygon(){}
12
13     protected:
14         RegularPolygon(){}
15     };
16 };
17 #endif
```

PS:基类必须提供无参构造函数，所以关于多边形的边长没有通过构造函数而是通过单独编写的 initialize 函数传参。

3.创建插件

在 xxx/include/xxx 下新建 C++ 头文件: polygon_plugins.h，内容如下:

```
1  #ifndef XXX_POLYGON_PLUGINS_H_
2  #define XXX_POLYGON_PLUGINS_H_
3  #include <xxx/polygon_base.h>
4  #include <cmath>
5
6  namespace polygon_plugins
7  {
8      class Triangle : public polygon_base::RegularPolygon
9      {
10     public:
11         Triangle(){}
12
13         void initialize(double side_length)
14         {
15             side_length_ = side_length;
16         }
17
18         double area()
19         {
20             return 0.5 * side_length_ * getHeight();
21         }
22
23         double getHeight()
24         {
25             return sqrt((side_length_ * side_length_) - ((side_length_ / 2) *
(side_length_ / 2)));
26         }
27
28     private:
29         double side_length_;
30     };
31
32     class Square : public polygon_base::RegularPolygon
33     {
34     public:
35         Square(){}
36
37         void initialize(double side_length)
38         {
39             side_length_ = side_length;
40         }
41
42         double area()
43         {
44             return side_length_ * side_length_;
```



```

45     }
46
47     private:
48         double side_length_;
49
50     };
51 };
52 #endif

```

该文件中创建了正方形与三角形两个衍生类继承基类。

4.注册插件

在 src 目录下新建 polygon_plugins.cpp 文件，内容如下：

```

1 //pluginlib 宏，可以注册插件类
2 #include <pluginlib/class_list_macros.h>
3 #include <xxx/polygon_base.h>
4 #include <xxx/polygon_plugins.h>
5
6 //参数1:衍生类 参数2:基类
7 PLUGINLIB_EXPORT_CLASS(polygon_plugins::Triangle,
8 polygon_base::RegularPolygon)
9
10 PLUGINLIB_EXPORT_CLASS(polygon_plugins::Square, polygon_base::RegularPolygon)

```

该文件会将两个衍生类注册为插件。

5.构建插件库

在 CMakeLists.txt 文件中设置内容如下：

```

1 include_directories(include)
2 add_library(polygon_plugins src/polygon_plugins.cpp)

```

至此，可以调用 catkin_make 编译，编译完成后，在工作空间/devel/lib目录下，会生成相关的 .so 文件。

6.使插件可用于ROS工具链

6.1配置xml

功能包下新建文件:polygon_plugins.xml,内容如下：

```

1 <!-- 插件库的相对路径 -->
2 <library path="lib/libpolygon_plugins">
3   <!-- type="插件类" base_class_type="基类" -->
4   <class type="polygon_plugins::Triangle"
base_class_type="polygon_base::RegularPolygon">
5     <!-- 描述信息 -->
6     <description>This is a triangle plugin.</description>
7   </class>
8   <class type="polygon_plugins::Square"
base_class_type="polygon_base::RegularPolygon">
9     <description>This is a square plugin.</description>
10  </class>
11 </library>

```

6.2 导出插件

package.xml文件中设置内容如下:

```

1 <export>
2   <xxx plugin="${prefix}/polygon_plugins.xml" />
3 </export>

```

标签的名称应与基类所属的功能包名称一致，plugin属性值为上一步中创建的xml文件。

编译后，可以调用 `rospack plugins --attrib=plugin xxx` 命令查看配置是否正常，如无异常，会返回.xml文件的完整路径，这意味着插件已经正确的集成到了ROS工具链。

7. 使用插件

src下新建c++文件: polygon_loader.cpp，内容如下:

```

1 //类加载器相关的头文件
2 #include <pluginlib/class_loader.h>
3 #include <xxx/polygon_base.h>
4
5 int main(int argc, char** argv)
6 {
7   //类加载器 -- 参数1:基类功能包名称 参数2:基类全限定名称
8   pluginlib::ClassLoader<polygon_base::RegularPolygon> poly_loader("xxx",
"polygon_base::RegularPolygon");
9
10  try
11  {
12    //创建插件类实例 -- 参数:插件类全限定名称
13    boost::shared_ptr<polygon_base::RegularPolygon> triangle =
poly_loader.createInstance("polygon_plugins::Triangle");
14    triangle->initialize(10.0);
15
16    boost::shared_ptr<polygon_base::RegularPolygon> square =
poly_loader.createInstance("polygon_plugins::Square");
17    square->initialize(10.0);
18

```

```

19     ROS_INFO("Triangle area: %.2f", triangle->area());
20     ROS_INFO("Square area: %.2f", square->area());
21 }
22 catch(pluginlib::PluginlibException& ex)
23 {
24     ROS_ERROR("The plugin failed to load for some reason. Error: %s",
ex.what());
25 }
26
27 return 0;
28 }

```

8.执行

修改CMakeLists.txt文件，内容如下：

```

1 add_executable(polygon_loader src/polygon_loader.cpp)
2 target_link_libraries(polygon_loader ${catkin_LIBRARIES})

```

编译然后执行: polygon_loader，结果如下：

```

1 [ INFO] [WallTime: 1279658450.869089666]: Triangle area: 43.30
2 [ INFO] [WallTime: 1279658450.869138007]: Square area: 100.00

```

10.4 nodelet

ROS通信是基于Node(节点)的，Node使用方便、易于扩展，可以满足ROS中大多数应用场景，但是也存在一些局限性，由于一个Node启动之后独占一根进程，不同Node之间数据交互其实是不同进程之间的数据交互，当传输类似于图片、点云的大容量数据时，会出现延时与阻塞的情况，比如：

现在需要编写一个相机驱动，在该驱动中有两个节点实现:其中节点A负责发布原始图像数据，节点B订阅原始图像数据并在图像上标注人脸。如果节点A与节点B仍按照之前实现，两个节点分别对应不同的进程，在两个进程之间传递容量可观图像数据，可能就会出现延时的情况，那么该如何优化呢？

ROS中给出的解决方案是:Nodelet，通过Nodelet可以将多个节点集成进一个进程。

概念

nodelet软件包旨在提供在同一进程中运行多个算法(节点)的方式，不同算法之间通过传递指向数据的指针来代替了数据本身的传输(类似于编程传值与传址的区别)，从而实现零成本的数据拷贝。

nodelet功能包的核心实现也是插件，是对插件的进一步封装：

- 不同算法被封装进插件类，可以像单独的节点一样运行；
- 在该功能包中提供插件类实现的基类:Nodelet；
- 并且提供了加载插件类的类加载器:NodeletLoader。

作用

应用于大容量数据传输的场景，提高节点间的数据交互效率，避免延时与阻塞。

另请参考：

- <http://wiki.ros.org/nodelet/>
- <http://wiki.ros.org/nodelet/Tutorials/Running%20a%20nodelet>
- https://github.com/ros/common_tutorials/tree/noetic-devel/nodelet_tutorial_math

10.4.1 使用演示

在ROS中内置了nodelet案例，我们先以该案例演示nodelet的基本使用语法，基本流程如下：

1. 案例简介；
2. nodelet基本使用语法；
3. 内置案例调用。

1.案例简介

以“ros- [ROS_DISTRO] -desktop-full”命令安装ROS时，nodelet默认被安装，如未安装，请调用如下命令自行安装：

```
1 | sudo apt install ros-<<ROS_DISTRO>>-nodelet-tutorial-math
```

在该案例中，定义了一个Nodelet插件类:Plus，这个节点可以订阅一个数字，并将订阅到的数字与参数服务器中的 value 参数相加后再发布。

需求:再同一线程中启动两个Plus节点A与B，向A发布一个数字，然后经A处理后，再发布并作为B的输入，最后打印B的输出。

2.nodelet 基本使用语法

使用语法如下：

```
1 | nodelet load pkg/Type manager - Launch a nodelet of type pkg/Type on manager manager
2 | nodelet standalone pkg/Type - Launch a nodelet of type pkg/Type in a standalone node
3 | nodelet unload name manager - Unload a nodelet a nodelet by name from manager
4 | nodelet manager - Launch a nodelet manager node
```

3.内置案例调用

1.启动roscore

```
1 | roscore
```

2.启动manager

```
1 | roslaunch nodelet_manager __name:=mymanager
```

__name:= 用于设置管理器名称。

3.添加nodelet节点

添加第一个节点:

```
1 | roslaunch nodelet_manager nodelet_load nodelet_tutorial_math/Plus mymanager __name:=n1
   | _value:=100
```

添加第二个节点:

```
1 | roslaunch nodelet_manager nodelet_load nodelet_tutorial_math/Plus mymanager __name:=n2
   | _value:=-50 /n2/in:=/n1/out
```

PS: 解释

```
roslaunch nodelet_manager nodelet_load nodelet_tutorial_math/Plus mymanager __name:=n1 _value:=100
```

1. roslaunch list 查看，nodelet 的节点名称是: /n1;
2. rostopic list 查看，订阅的话题是: /n1/in，发布的话题是: /n1/out;
3. rosparam list查看，参数名称是: /n1/value。

```
roslaunch nodelet_manager nodelet_standalone nodelet_tutorial_math/Plus mymanager __name:=n2
_value:=-50 /n2/in:=/n1/out
```

1. 第二个nodelet 与第一个同理;
2. 第二个nodelet 订阅的话题由 /n2/in 重映射为 /n1/out。

优化:也可以将上述实现集成进launch文件:

```
1 | <launch>
2 |   <!-- 设置nodelet管理器 -->
3 |   <node pkg="nodelet" type="nodelet" name="mymanager" args="manager"
   |   output="screen" />
4 |   <!-- 启动节点1, 名称为 n1, 参数 /n1/value 为100 -->
5 |   <node pkg="nodelet" type="nodelet" name="n1" args="load
   |   nodelet_tutorial_math/Plus mymanager" output="screen" >
6 |     <param name="value" value="100" />
7 |   </node>
8 |   <!-- 启动节点2, 名称为 n2, 参数 /n2/value 为-50 -->
9 |   <node pkg="nodelet" type="nodelet" name="n2" args="load
   |   nodelet_tutorial_math/Plus mymanager" output="screen" >
10 |     <param name="value" value="-50" />
11 |     <remap from="/n2/in" to="/n1/out" />
12 |   </node>
13 |
14 | </launch>
```

4.执行

向节点n1发布消息:

```
1 | rostopic pub -r 10 /n1/in std_msgs/Float64 "data: 10.0"
```

打印节点n2发布的消息:

```
1 | rostopic echo /n2/out
```

最终输出结果应该是:60。

10.4.2 nodelet实现

nodelet本质也是插件，实现流程与插件实现流程类似，并且更为简单，不需要自定义接口，也不需要使用类加载器加载插件类。

需求:参考 nodelet 案例，编写 nodelet 插件类，可以订阅输入数据，设置参数，发布订阅数据与参数相加的结果。

流程:

1. 准备;
2. 创建插件类并注册插件;
3. 构建插件库;
4. 使插件可用于ROS工具链;
5. 执行。

1.准备

新建功能包，导入依赖:roscpp、nodelet;

2.创建插件类并注册插件

```
1 | #include "nodelet/nodelet.h"
2 | #include "pluginlib/class_list_macros.h"
3 | #include "ros/ros.h"
4 | #include "std_msgs/Float64.h"
5 |
6 | namespace nodelet_demo_ns {
7 | class MyPlus: public nodelet::Nodelet {
8 |     public:
9 |         MyPlus(){
10 |             value = 0.0;
11 |         }
12 |         void onInit(){
13 |             //获取 NodeHandle
14 |             ros::NodeHandle& nh = getPrivateNodeHandle();
15 |             //从参数服务器获取参数
16 |             nh.getParam("value",value);
17 |             //创建发布与订阅对象
```

```

18         pub = nh.advertise<std_msgs::Float64>("out",100);
19         sub = nh.subscribe<std_msgs::Float64>("in",100,&MyPlus::doCb,this);
20
21     }
22     //回调函数
23     void doCb(const std_msgs::Float64::ConstPtr& p){
24         double num = p->data;
25         //数据处理
26         double result = num + value;
27         std_msgs::Float64 r;
28         r.data = result;
29         //发布
30         pub.publish(r);
31     }
32     private:
33     ros::Publisher pub;
34     ros::Subscriber sub;
35     double value;
36
37 };
38 }
39 PLUGINLIB_EXPORT_CLASS(nodelet_demo_ns::MyPlus,nodelet::Nodelet)

```

3.构建插件库

CMakeLists.txt配置如下:

```

1  ...
2  add_library(mynodeletlib
3      src/myplus.cpp
4  )
5  ...
6  target_link_libraries(mynodeletlib
7      ${catkin_LIBRARIES}
8  )

```

编译后, 会在 `工作空间/devel/lib/` 先生成文件: libmynodeletlib.so。

4.使插件可用于ROS工具链

4.1配置xml

新建 xml 文件, 名称自定义(比如:my_plus.xml), 内容如下:

```

1  <library path="lib/libmynodeletlib">
2      <class name="demo04_nodelet/MyPlus" type="nodelet_demo_ns::MyPlus"
3      base_class_type="nodelet::Nodelet" >
4          <description>hello</description>
5      </class>
6  </library>

```

4.2 导出插件

```
1 <export>
2   <!-- Other tools can request additional information be placed here -->
3   <nodelet plugin="${prefix}/my_plus.xml" />
4 </export>
```

5. 执行

可以通过launch文件执行nodelet，示例内容如下：

```
1 <launch>
2   <node pkg="nodelet" type="nodelet" name="my" args="manager"
   output="screen" />
3   <node pkg="nodelet" type="nodelet" name="p1" args="load
   demo04_nodelet/MyPlus my" output="screen">
4     <param name="value" value="100" />
5     <remap from="/p1/out" to="con" />
6   </node>
7   <node pkg="nodelet" type="nodelet" name="p2" args="load
   demo04_nodelet/MyPlus my" output="screen">
8     <param name="value" value="-50" />
9     <remap from="/p2/in" to="con" />
10  </node>
11
12 </launch>
```

运行launch文件，可以参考上一节方式向 p1发布数据，并订阅p2输出的数据，最终运行结果也与上一节类似。

10.5 本章小结

本章介绍了ROS中的一些进阶内容，主要内容如下：

- Action 通信；
- 动态参数；
- pluginlib；
- nodelet。

上述内容其实都是对之前通信机制缺陷的进一步优化：**action**较之于以往的服务通信是带有连续反馈的，更适用于耗时的请求响应场景；动态参数较之于参数服务器实现，则可以保证参数读取的实时性；最后，**nodelet**可以动态加载多个节点到同一进程，不再是一个节点独占一个进程，从而可以零成本的实现不同节点之间的数据交互，降低了数据传输的延时，提高了数据传输的效率；当然，**nodelet**是插件的应用之一，所以在介绍 **nodelet** 之前，我们又先学习了 **pluginlib**，借助 **pluginlib** 可以实现可插拔的设计，让程序更为灵活、易于扩展且方便维护。

